

Concurrent Programming

Outline

- Topics:
 - Shared variables
 - Synchronizing with semaphores
 - Producer-Consumer Problem
 - Readers-Writers Problem
 - Concurrent Server Based on Pthreading
 - Concurrent Issues
- Suggested reading:
 - 12.4~12.7

A process with multiple threads

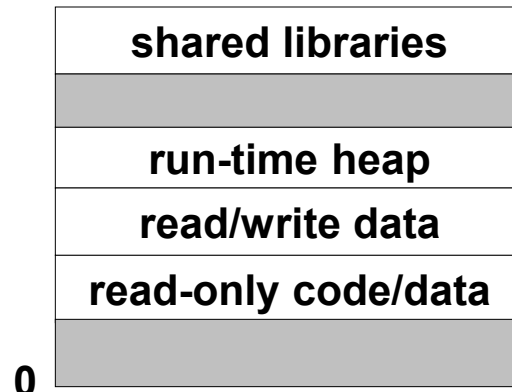
Thread 1 (main thread)

stack 1

Thread 1 context:

Data registers
Condition codes
SP1
PC1

Shared code and data



Kernel context:

VM structures
Descriptor table
brk pointer

Thread 2 (peer thread)

stack 2

Thread 2 context:

Data registers
Condition codes
SP2
PC2

Shared variables in threaded C programs

- Which variables in a C program are **shared** or **not**?
 - What is the memory model for threads?
 - How are instances of the variable mapped to memory?
 - How many threads reference each of these instances?

Threads memory model

- Conceptual model:
 - Each thread runs in the context of a process
 - Each thread has its own separate thread context
 - Thread ID, stack, stack pointer, program counter, condition codes, and general purpose registers
 - All threads share the remaining process context
 - Code, data, heap, and shared library segments of the process virtual address space
 - Open files and installed handlers

Threads memory model

- Operationally, this model is not strictly enforced:
 - Register values are truly separate and protected
 - However, any thread can read and write the stack of any other thread

The **MISMATCH** between the **CONCEPTUAL** and **OPERATION MODEL** is a source of confusion and errors

Shared variable analysis

<pre>7 int main() 8 { 9 int i; 10 pthread_t tid; 11 char *msgs[N] = { 12 "Hello from foo", 13 "Hello from bar" 14 }; 15 16 ptr = msgs; 17 for (i = 0; i < N; i++) 18 Pthread_create(&tid, NULL, thread, (void *)i); 19 Pthread_exit(NULL); 20 }</pre>	<pre>1 #include "csapp.h" 2 #define N 2 3 void *thread(void *vargp); 4 5 char **ptr; 6 /* global variable */</pre>
---	---

Shared variable analysis

```
21 void *thread(void *vargp)
22 {
23     int myid = (int)vargp;
24     static int cnt = 0;
25
26     printf("[%d]:%s(cnt=%d)\n", myid, ptr[myid], ++cnt);
27 }
```


Mapping Variable Instances to Memory

- Global variables
 - Def: Variable declared outside of a function
 - Virtual memory contains exactly one instance of any global variable
- Local variables
 - Def: Variable declared inside function without `static` attribute
 - Each thread stack contains one instance of each local variable
- Local static variables
 - Def: Variable declared inside function with the `static` attribute
 - Virtual memory contains exactly one instance of any local static variable

Shared variable analysis

- Which variables are shared?

Variable instance	Referenced by main thread?	Referenced by peer thread-0?	Referenced by peer thread-1?
ptr	yes	yes	yes
cnt	no	yes	yes
i.m	yes	no	no
msgs.m	yes	yes	yes
myid.p0	no	yes	no
myid.p1	no	no	yes

Shared variable analysis

- Answer: A variable x is shared iff multiple threads **reference** at least one instance of x
- Thus:
 - `ptr`, `cnt`, and `msgs` are shared
 - `i` and `myid` are NOT shared

Q: Is there any bug in the above code?

Shared variable analysis

```
9  int main()
10 {
11     pthread_t tid1, tid2;
12
13     Pthread_create(&tid1, NULL, count, NULL);
14     Pthread_create(&tid2, NULL, count, NULL);
15     Pthread_join(tid1, NULL);
16     Pthread_join(tid2, NULL);
17
18     if (cnt != (unsigned)NITERS*2)
19         printf("BOOM! cnt=%d\n", cnt);
20     else
21         printf("OK cnt=%d\n", cnt);
22     exit(0);
23 }
```

```
1  #include "csapp.h"
2
3  #define NITERS 100000000
4  void *count(void *arg);
5
6  /* shared variable */
7  unsigned int cnt = 0;
8
```

Shared variable analysis

```
24
25 /* thread routine */
26 void *count(void *arg)
27 {
28     int i;
29     for (i = 0; i < NITERS; i++)
30         cnt++;
31     return NULL;
32 }
```

Shared variable analysis

```
linux> badcnt  
BOOM! cnt=198841183
```

```
linux> badcnt  
BOOM! cnt=198261801
```

```
linux> badcnt  
BOOM! cnt=198269672
```

- **cnt** should be equal to 200,000,000.
- What went wrong?!

Assembly code for counter loop

C code for thread i

```
for (i = 0; i < NITERS; i++)  
    cnt++;
```

Assembly code for counter loop

Asm code for thread i

Head (H_i)

```
movq (%rdi), %rcx
testq %rcx, %rcx
jle .L2
movl $0, %eax
```

Load cnt (L_i)

Update cnt (U_i)

Store cnt (S_i)

.L3:

```
movq cnt(%rip), %rdx # Load
addq %rdx             # Update
movq %rdx, cnt(%rip) # Store
```

Tail (T_i)

```
addq $1, %rax
cmpq %rcx, %rax
jne .L3
```

.L2:

Concurrent execution

- Key idea: In general, any sequentially consistent interleaving is possible, but some are incorrect!
 - I_i denotes that thread i executes instruction I
 - $\%rdx_i$ is the contents of $\%rdx$ in thread i 's context

Concurrent execution

i (thread)	instr_i	$\%rdx_1$	$\%rdx_2$	cnt
1	H_1	-	-	0
1	L_1	0	-	0
1	U_1	1	-	0
1	S_1	1	-	1
2	H_2	-	-	1
2	L_2	-	1	1
2	U_2	-	2	1
2	S_2	-	2	2
2	T_2	-	2	2
1	T_1	1	-	2

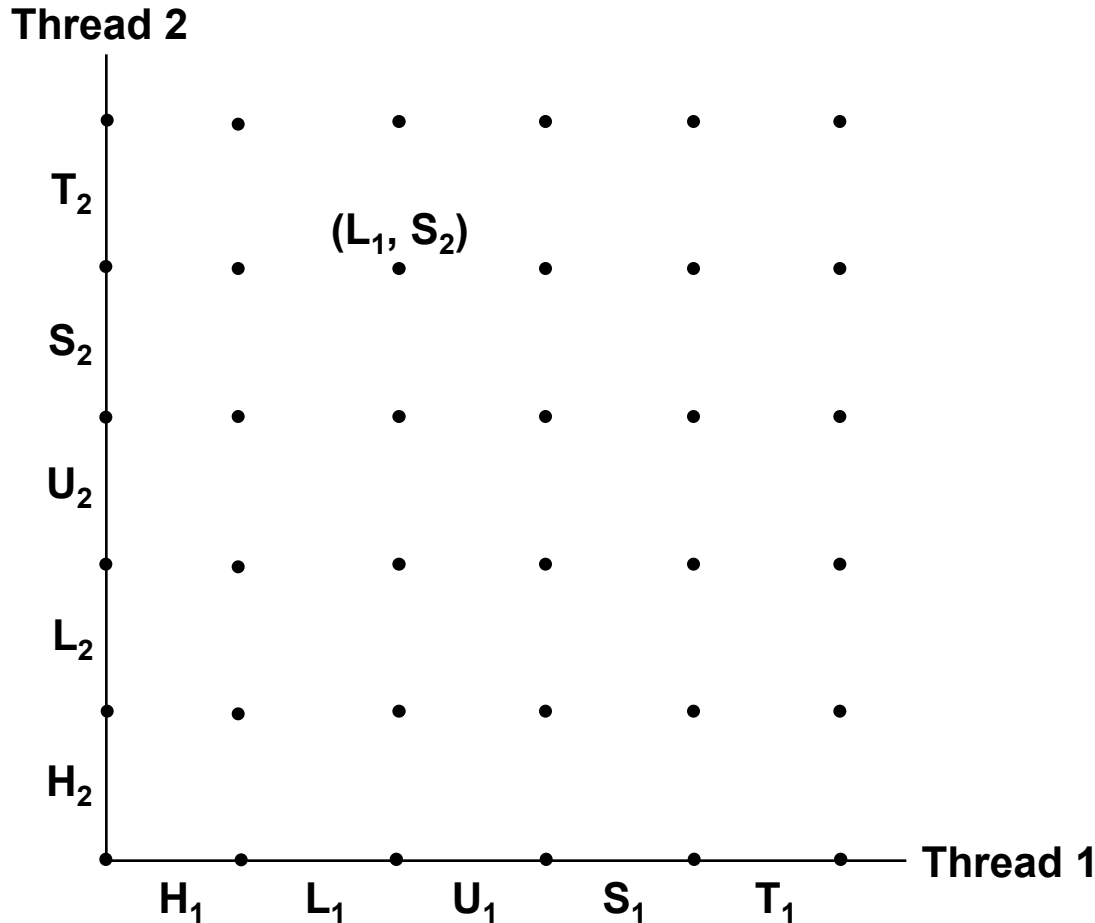
OK

Concurrent execution (cont)

- Incorrect ordering: two threads increment the counter, but the result is 1 instead of 2

i (thread)	instr _i	%rdx ₁	%rdx ₂	cnt
1	H ₁	-	-	0
1	L ₁	0	-	0
1	U ₁	1	-	0
2	H ₂	-	-	0
2	L ₂	-	0	0
1	S ₁	1	-	1
1	T ₁	1	-	1
2	U ₂	-	1	1
2	S ₂	-	1	1
2	T ₂	-	1	1

Progress graphs



A *progress graph* depicts the discrete *execution state space* of concurrent threads.

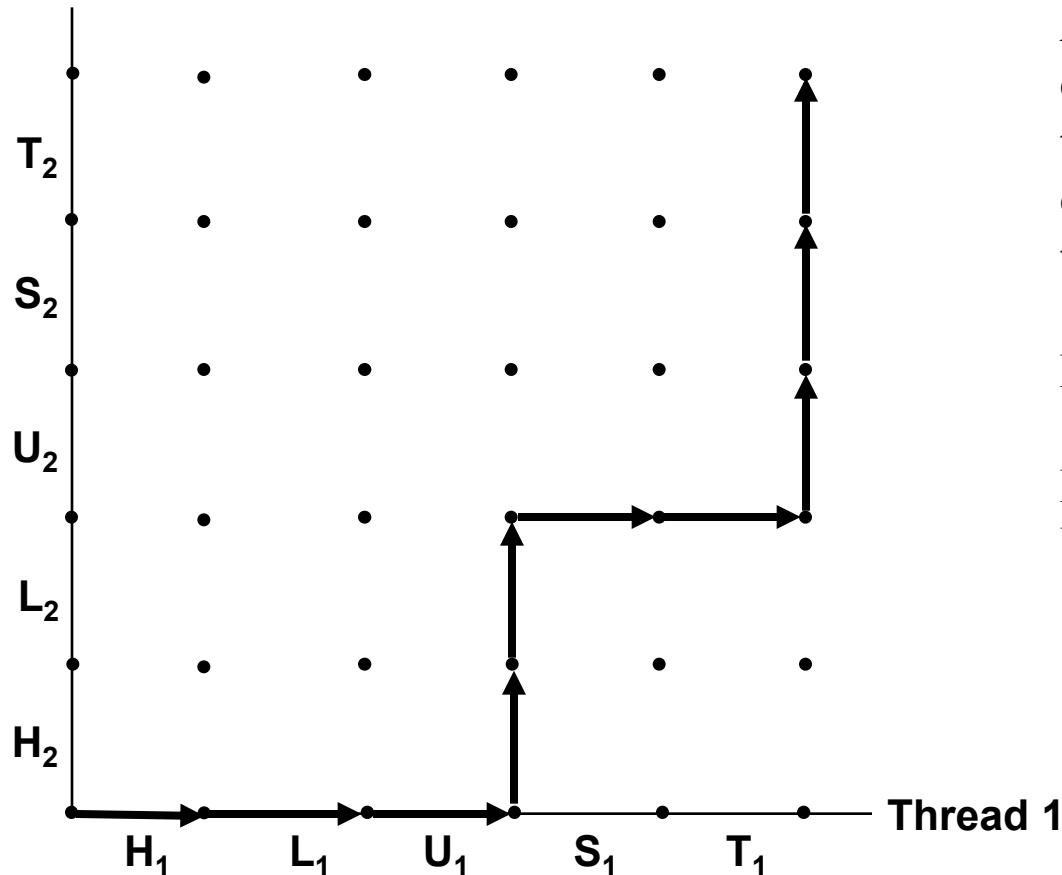
Each axis corresponds to the sequential order of instructions in a thread.

Each point corresponds to a possible *execution state* (Inst₁, Inst₂).

E.g., (L₁, S₂) denotes state where thread 1 has completed L₁ and thread 2 has completed S₂.

Trajectories in progress graphs

Thread 2

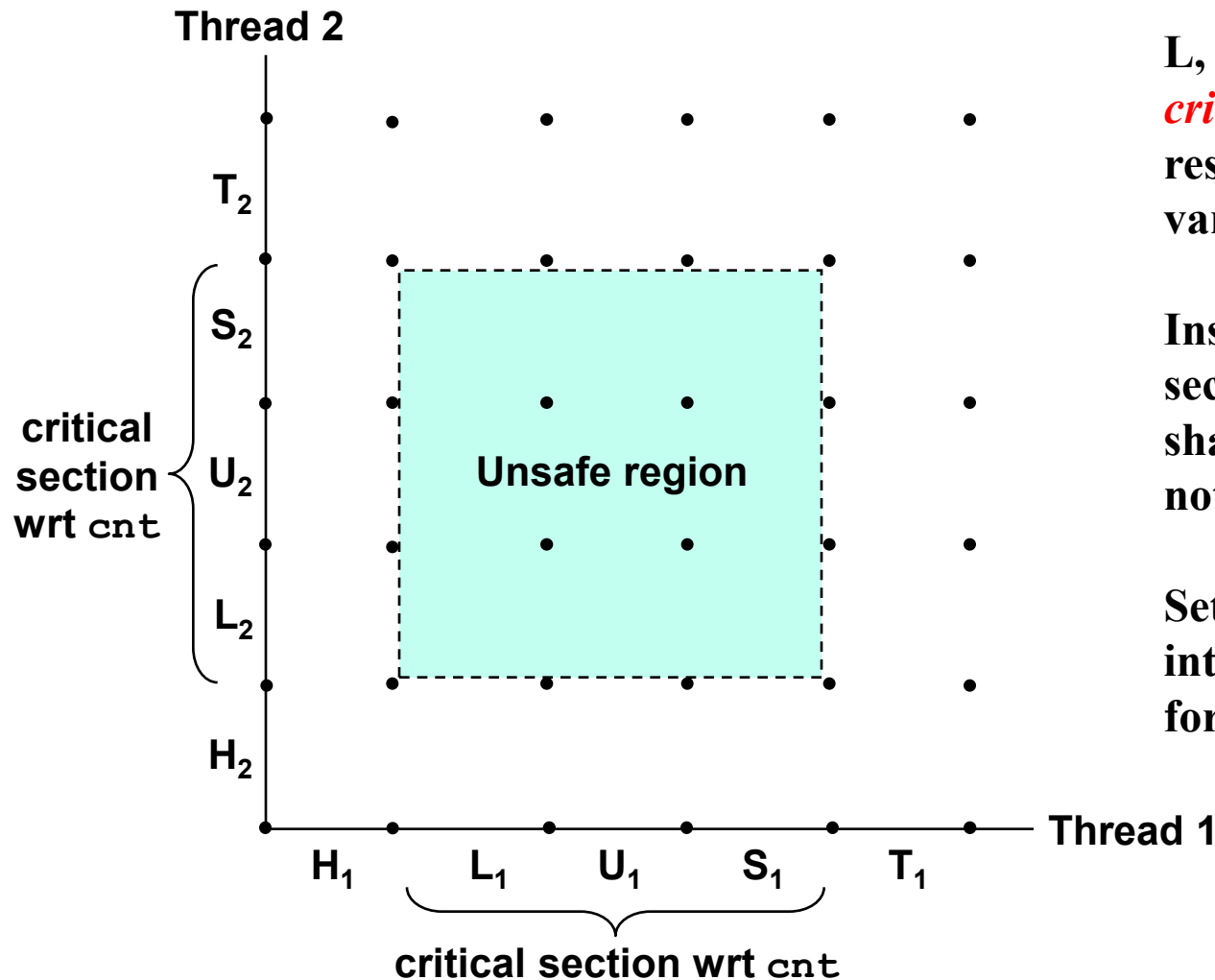


A *trajectory* is a sequence of legal state transitions that describes one possible concurrent execution of the threads.

Example:

H1, L1, U1, H2, L2, S1, T1, U2, S2, T2

Critical sections and unsafe regions

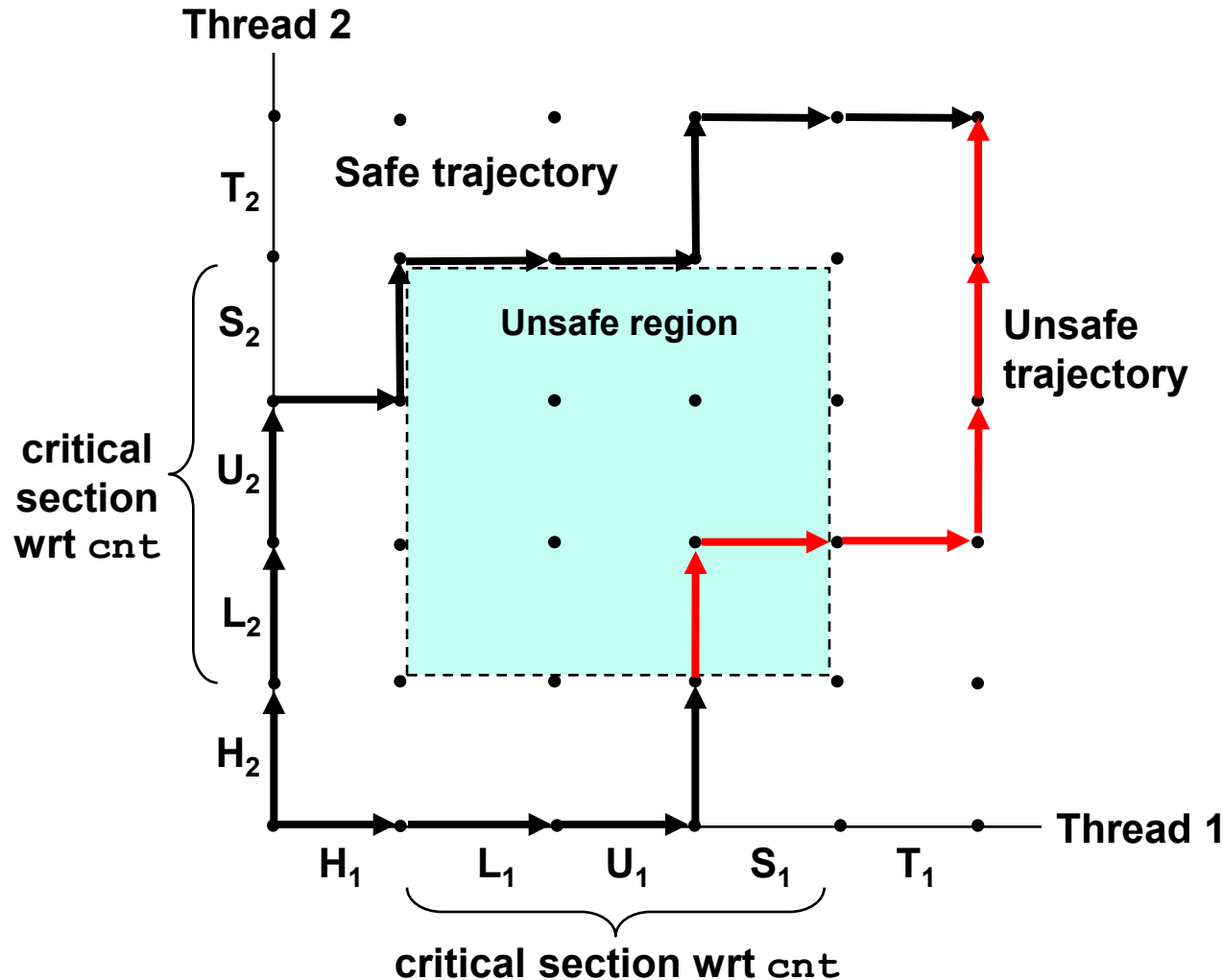


L , U , and S form a **critical section** with respect to the shared variable `cnt`.

Instructions in critical sections (write to some shared variable) should not be **interleaved**.

Sets of states where such interleaving occurs form **unsafe regions**.

Safe and unsafe trajectories



Def: A trajectory is **safe** iff it doesn't touch any part of an unsafe region.

Claim: A trajectory is **correct** (write cnt) iff it is safe.

Synchronizing with semaphores

- Dijkstra's P and V operations on **semaphores**
 - semaphore: non-negative integer synchronization variable
 - $P(s)$: [**while (s == 0) wait(); s--;**]
 - Dutch for "Proberen" (test)
 - $V(s)$: [**s++;**]
 - Dutch for "Verhogen" (increment)

[**Just pseudo-code**]

Synchronizing with semaphores

- Dijkstra's P and V operations on **semaphores**
 - OS guarantees that operations between brackets [] are executed **indivisibly**
 - Only one P or V operation at a time can modify s
 - When `while` loop in P terminates, only that P can decrement s
- Semaphore invariant: $(s \geq 0)$
- V will wake up blocked P
 - No ordering is ensured

POSIX semaphores

```
#include <semaphore.h>

int sem_init(sem_t *sem, 0, unsigned int value);
int sem_wait(sem_t *s); /* P(s) */
int sem_post(sem_t *s); /* V(s) */

#include "csapp.h"

void P(sem_t *s); /* Wrapper function for sem_wait */
void V(sem_t *s); /* Wrapper function for sem_wait */
```

Sharing with POSIX semaphores

```
#include "csapp.h"
#define NITERS 100000000
unsigned int cnt; /* counter */
sem_t sem;      /* semaphore */

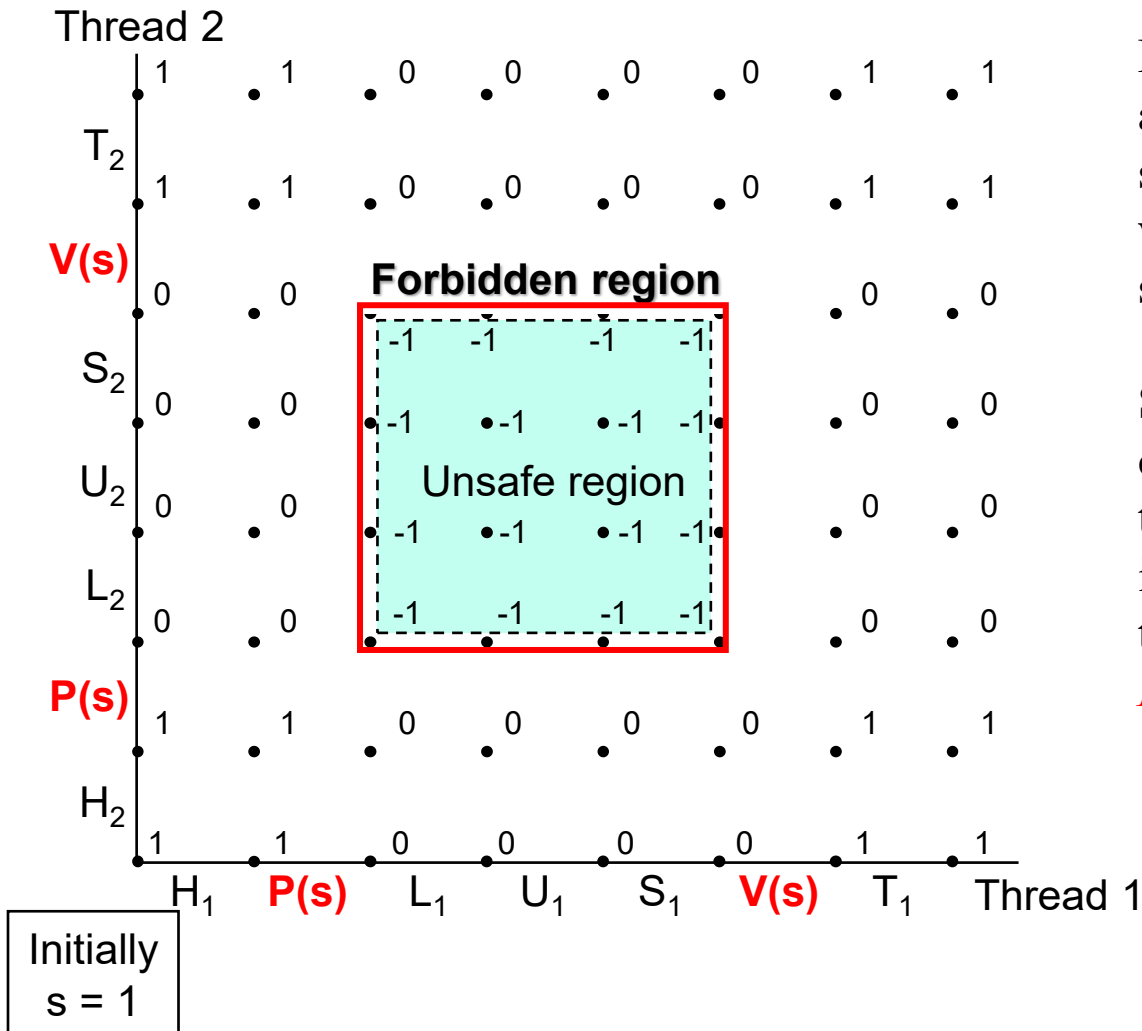
int main() {
    pthread_t tid1, tid2;
    Sem_init(&sem, 0, 1);
    /* create 2 threads and wait */
    ...
    if (cnt != (unsigned)NITERS*2)
        printf("BOOM! cnt=%d\n", cnt);
    else
        printf("OK cnt=%d\n", cnt);
    exit(0);
}
```

Sharing with POSIX semaphores

```
/* thread routine */
void *count(void *arg)
{
    int i;

    for (i=0; i<NITERS; i++) {
        P(&sem);
        cnt++;
        V(&sem);
    }
    return NULL;
}
```

Safe sharing with semaphores



Provide *mutually exclusive* access to shared variable by surrounding critical section with P and V operations on semaphore s (initially set to 1).

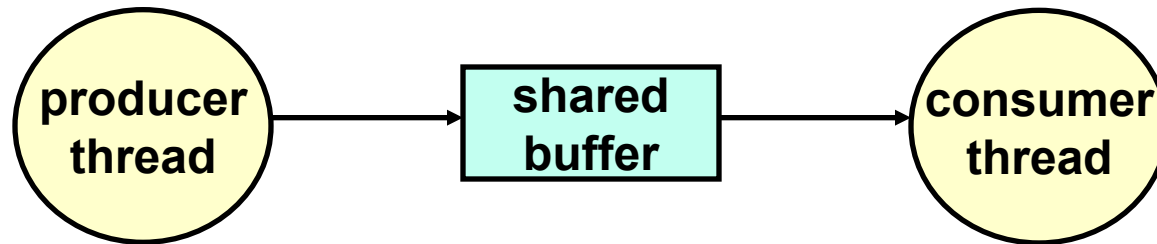
Semaphore invariant creates a *forbidden region* that encloses unsafe region and is never touched by any trajectory.

Mutual exclusion

Using Semaphores to Schedule Access to Shared Resources

- Basic idea: Thread uses a semaphore operation to notify another thread that some condition has become true
 - Use **counting semaphores** to keep track of resource state
 - Use **binary semaphores (mutex)** to notify other threads
- Two classic examples:
 - The Producer-Consumer Problem
 - The Readers-Writers Problem

Signaling with semaphores



- Common synchronization pattern:
 - Producer waits for empty slot, inserts item in buffer, and notifies consumer
 - Consumer waits for item, removes it from buffer, and notifies producer

Producer-Consumer on 1-element Buffer

```
int main() {
    pthread_t tid_producer;
    pthread_t tid_consumer;

    /* Initialize the semaphores */
    Sem_init(&shared.empty, 0, 1);
    Sem_init(&shared.full, 0, 0);
    /* Create threads and wait */
    Pthread_create(&tid_producer, NULL, producer, NULL);
    Pthread_create(&tid_consumer, NULL, consumer, NULL);
    Pthread_join(tid_producer, NULL);
    Pthread_join(tid_consumer, NULL);

    exit(0);
}
```

```
#define NITERS 5
```

```
struct {
    int buf;
    sem_t full;
    sem_t empty;
} shared;
```


Producer-Consumer on 1-element Buffer

Initially: `empty==1`, `full==0`

Producer Thread

```
void *producer(void *arg) {
    int i, item;

    for (i = 0; i < NITERS; i++) {
        /* Produce item */
        item = i;
        printf("produced %d\n", item);

        /* Write item to buf */
        P(&shared.empty);
        shared.buf = item;
        V(&shared.full);
    }
    return NULL;
}
```

Consumer Thread

```
void *consumer(void *arg) {
    int i, item;

    for (i = 0; i < NITERS; i++) {
        /* Read item from buf */
        P(&shared.full);
        item = shared.buf;
        V(&shared.empty);

        /* Consume item */
        printf("consumed %d\n", item);
    }
    return NULL;
}
```

Readers-Writers Problem

- Generalization of the mutual exclusion problem
- Problem statement:
 - Reader threads only read the object
 - Writer threads modify the object
 - Writers must have **exclusive** access to the object
 - Unlimited number of readers can access the object

Readers-Writers Problem

- Occurs frequently in real systems, e.g.,
 - Online airline reservation system
 - Multithreaded caching Web proxy

Variants of Readers-Writers

- First readers-writers problem (favors readers)
 - No reader should be kept waiting unless a writer has already been granted permission to use the object
 - A reader that arrives after a waiting writer gets priority over the writer

Variants of Readers-Writers

- Second readers-writers problem (favors writers)
 - Once a writer is ready to write, it performs its write as soon as possible
 - A reader that arrives after a writer must wait, even if the writer is also waiting
- **Starvation** (where a thread waits indefinitely) is possible in both cases

Solution to First Readers-Writers Problem

Readers:

```
int readcnt;      /* Initially 0 */
sem_t mutex, w; /* Both initially 1 */

void reader(void) {
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* First in */
            P(&w);
        V(&mutex);

        /* Reading happens here */

        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&mutex);
    }
}
```

Writers:

```
void writer(void) {
    while (1) {
        P(&w);

        /* Writing here */

        V(&w);
    }
}
```

- Q1. How avoiding W-W conflict?
- Q2. How avoiding R-W conflict?
- Q3. How supporting R-R concurrency?
- Q4. How achieving Reader first?

Solution to Second Readers-Writers Problem

```
typedef struct {
    volatile int readers;
    volatile int writers;
    volatile int waiting_writers;
    pthread_mutex_t mutex;
} rwlock_t;

void rwlock_init(rwlock_t *lock) {
    lock->readers = 0;
    lock->writers = 0;
    lock->waiting_writers = 0;
    pthread_mutex_init(&lock->mutex, NULL);
}
```

Solution to Second Readers-Writers Problem

```
void rwlock_acquire_readlock(rwlock_t *lock) {
    pthread_mutex_lock(&lock->mutex);
    while (lock->writers > 0 || lock->waiting_writers > 0) {
        pthread_mutex_unlock(&lock->mutex);
        usleep(1000); // 让出CPU并稍作等待
        pthread_mutex_lock(&lock->mutex);
    }
    lock->readers++;
    pthread_mutex_unlock(&lock->mutex);
}

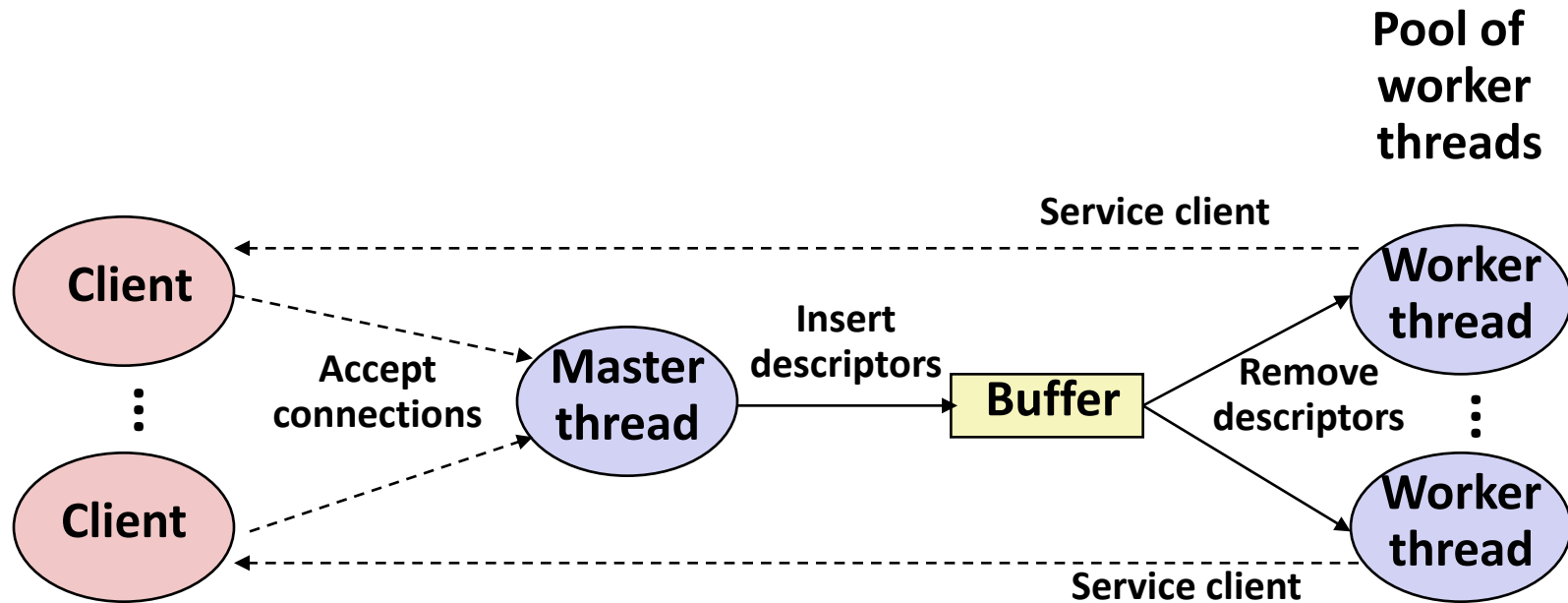
void rwlock_release_readlock(rwlock_t *lock) {
    pthread_mutex_lock(&lock->mutex);
    lock->readers--;
    pthread_mutex_unlock(&lock->mutex);
}
```


Solution to Second Readers-Writers Problem

```
void rwlock_acquire_writelock(rwlock_t *lock) {
    pthread_mutex_lock(&lock->mutex);
    lock->waiting_writers++;
    while (lock->readers > 0 || lock->writers > 0) {
        pthread_mutex_unlock(&lock->mutex);
        usleep(1000); // 让出CPU并稍作等待
        pthread_mutex_lock(&lock->mutex);
    }
    lock->waiting_writers--;
    lock->writers++;
    pthread_mutex_unlock(&lock->mutex);
}

void rwlock_release_writelock(rwlock_t *lock) {
    pthread_mutex_lock(&lock->mutex);
    lock->writers--;
    pthread_mutex_unlock(&lock->mutex);
}
```

Case Study: Prethreaded Concurrent Server



Q1: How to set the number of worker threads?

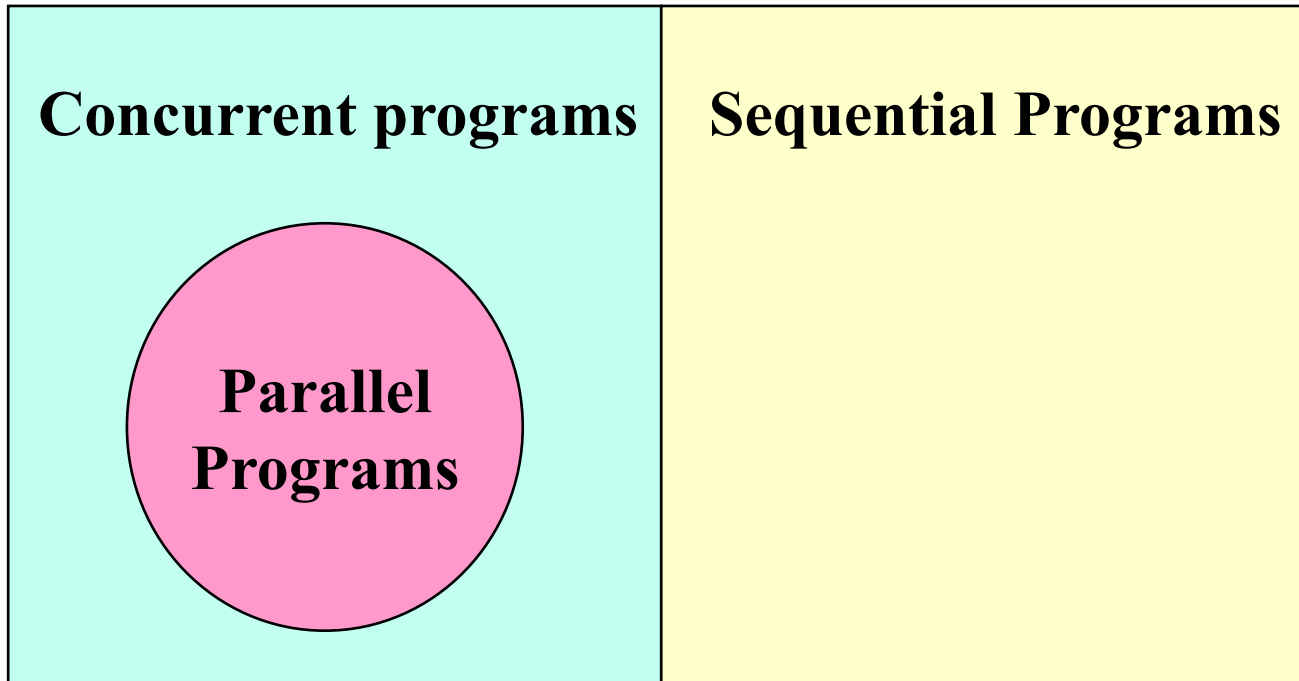
Q2: Prethreading also exists in Android/HarmonyOS. For what?

Tip: You should try to vibe this server.

MULTI-THREADED

Using Threads for Parallelism

All programs



```

1  #include "csapp.h"
2  #define MAXTHREADS 32
3
4  void *sum_mutex(void *vargp); /* Thread routine */
5
6  /* Global shared variables */
7  long gsum; /* Global sum */
8  long nelems_per_thread; /* Number of elements summed by each thread */
9  sem_t mutex; /* Mutex to protect global sum */
10
11 int main(int argc, char **argv)
12 {
13     long i, nelems, log_nelems, nthreads, myid[MAXTHREADS];
14     pthread_t tid[MAXTHREADS];
15
16     /* Get input arguments */
17     if (argc != 3) {
18         printf("Usage: %s <nthreads> <log_nelems>\n", argv[0]);
19         exit(0);
20     }

```

```
21     nthreads = atoi(argv[1]);
22     log_nelems = atoi(argv[2]);
23     nelems = (1L << log_nelems);
24     nelems_per_thread = nelems / nthreads;
25     sem_init(&mutex, 0, 1);
26
27     /* Create peer threads and wait for them to finish */
28     for (i = 0; i < nthreads; i++) {
29         myid[i] = i;
30         Pthread_create(&tid[i], NULL, sum_mutex, &myid[i]);
31     }
32     for (i = 0; i < nthreads; i++)
33         Pthread_join(tid[i], NULL);
34
35     /* Check final answer */
36     if (gsum != (nelems * (nelems-1))/2)
37         printf("Error: result=%ld\n", gsum);
38
39     exit(0);
40 }
```

```

1 /* Thread routine for psum-mutex.c */
2 void *sum_mutex(void *vargp)
3 {
4     int myid = *((int *)vargp);          /* Extract the thread ID */
5     long start = myid * nelems_per_thread; /* Start element index */
6     long end = start + nelems_per_thread; /* End element index */
7     long i;
8
9     for (i = start; i < end; i++) {
10         p(&mutex) ;
11         gsum += i;
12         V(&mutex);
13     }
14     return NULL;
15 }

```

Running time in seconds

Version	Number of threads				
	1	2	4	8	16
psum-mutex	68	432	719	552	599

Run the code on four cores (how to explain the perf?)

- Synchronization overhead is expensive and should be avoided if possible
- If it cannot be avoided, the overhead should be amortized by as much useful computation as possible

Q: How to optimize the performance?

```

1 /* Thread routine for psum-array.c */
2 void *sum_array(void *vargp)
3 {
4     int myid = *((int *)vargp);          /* Extract the thread ID */
5     long start = myid * nelems_per_thread; /* Start element index */
6     long end = start + nelems_per_thread; /* End element index */
7     long i;
8
9     for (i = start; i < end; i++) {
10         psum[myid] += i ;
11     }
12     return NULL;
13 }

```

Version	Number of threads				
	1	2	4	8	16
psum-mutex	68.00	432.00	719.00	552.00	599.00
psum-array	7.26	3.64	1.91	1.85	1.84

The main thread (not shown) defines a global array called psum.

Each peer thread accumulates its partial sum in psum[i]. After the peer threads have terminated, the main thread sums up the elements of the psum vector.

Q: How to further improve the performance?

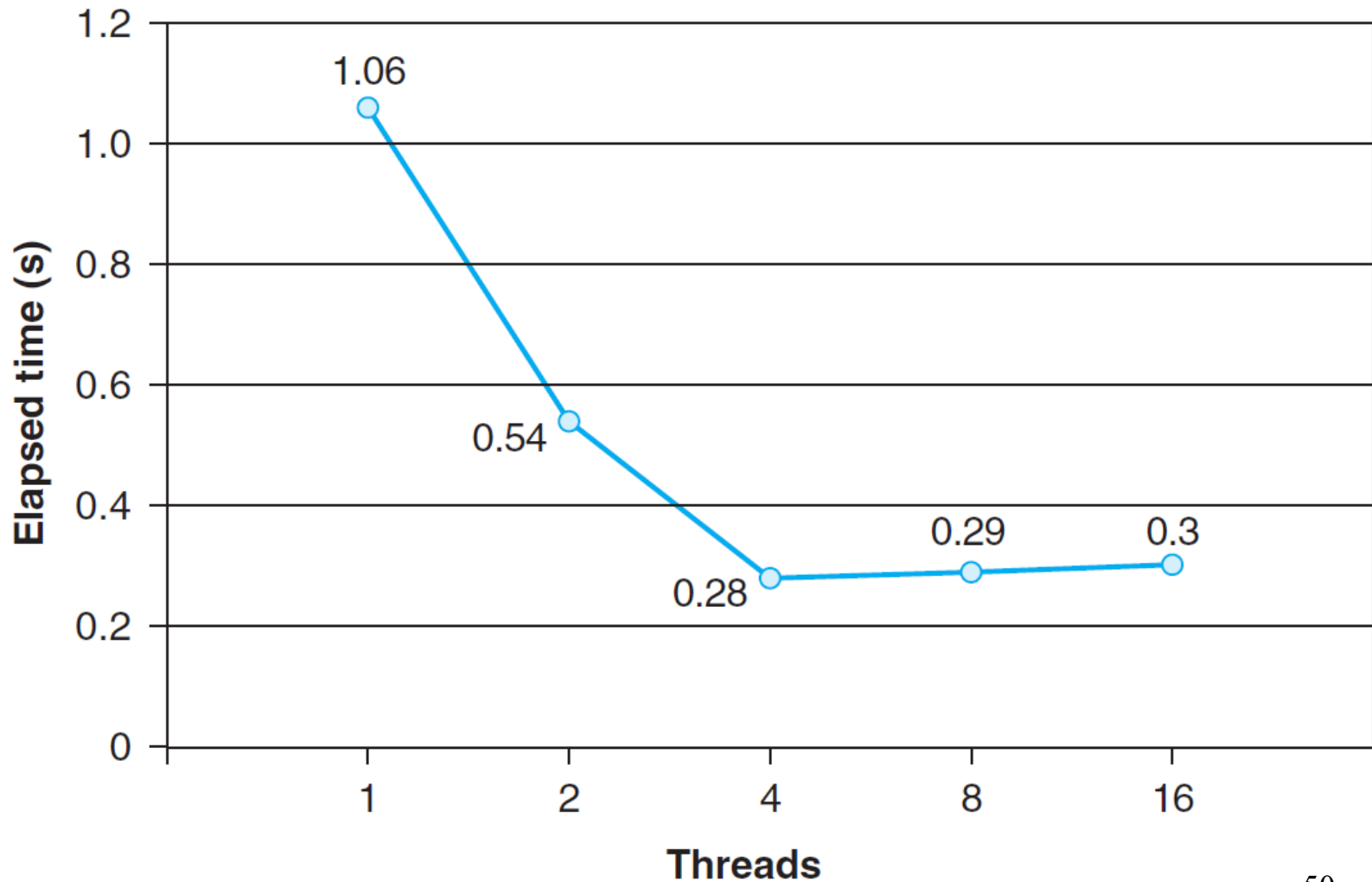

```

1 /* Thread routine for psum-local.c*/
2 void *sum_array(void *vargp)
3 {
4     int myid = *((int *)vargp);          /* Extract the thread ID */
5     long start = myid * nelems_per_thread; /* Start element index */
6     long end = start + nelems_per_thread; /* End element index */
7     long i, sum = 0;
8
9     for (i = start; i < end; i++) {
10         sum += i;
11     }
12     psum[myid] = sum ;
13     return NULL;
14 }/

```

Version	Number of threads				
	1	2	4	8	16
psum-mutex	68.00	432.00	719.00	552.00	599.00
psum-array	7.26	3.64	1.91	1.85	1.84
psum-local	1.06	0.54	0.28	0.29	0.30

Using Threads for Parallelism



(Relative) Speedup = T_1/T_p

Thread-safe functions

- Functions called from a thread must be **thread-safe**
- Definition: A function is thread-safe iff it will always produce correct results when called **repeatedly** from multiple concurrent threads

Thread-safe functions

- We identify four (non-disjoint) classes of thread-unsafe functions:
 - Class 1: Failing to protect shared variables
 - Class 2: Relying on persistent state across invocations
 - Class 3: Returning a pointer to a static variable
 - Class 4: Calling thread-unsafe functions

Thread-unsafe functions

- Class 1: Failing to protect shared variables
 - Fix: use Pthreads P/V operations
 - Do not need to modify the calling code
 - Issue: synchronization operations will slow down code

Thread-unsafe functions (case 2)

- Class 2: Relying on persistent state across invocations

```
unsigned int next = 1;
/* rand - return pseudo-random int on 0..32767 */
int rand(void)
{
    next = next * 110351524 + 12345 ;
    return (unsigned int) ((next/65536) % 32768) ;
}
/* srand - set seed for rand() */
void srand(unsigned int seed)
{
    next = seed;
}
```

Thread-unsafe functions (case 2)

- Pass state as part of argument
 - and, thereby, eliminate static state

```
/* rand_r - return pseudo-random int on 0..32767 */  
int rand_r(int *nextp)  
{  
    *nextp = *nextp * 110351524 + 12345 ;  
    return (unsigned int) ((*nextp/65536) % 32768) ;  
}
```

Thread-unsafe functions (case 3)

- Class 3: Returning a pointer to a static variable

```
char *ctime(const time_t *timep)
{
    static char *p;
    <get current time and converted to string>
    return &p;
}
```


Thread-safe functions

- Class 3: (Fixes)
 - Rewrite: caller passes pointer to struct
 - Issue: Requires changes in callee
 - "Lock-and-copy"

```
privatep = Malloc(...);  
ctime_ts(&t, privatep);
```

```
char *ctime_ts (const time_t timep, char *privatep)  
{  
    char *sharedp;  
  
    P(&mutex);  
    sharedp = ctime(timep);  
    strcpy(privatep, sharedp) ;  
    V(&mutex);  
    return privatep;  
}
```

Thread-unsafe functions (case 4)

- Class 4: Calling thread-unsafe functions
 - Calling one thread-unsafe function makes the entire function that calls it thread-unsafe
 - Actually, it depends
 - Fix: Modify the function so it calls only thread-safe functions 😊

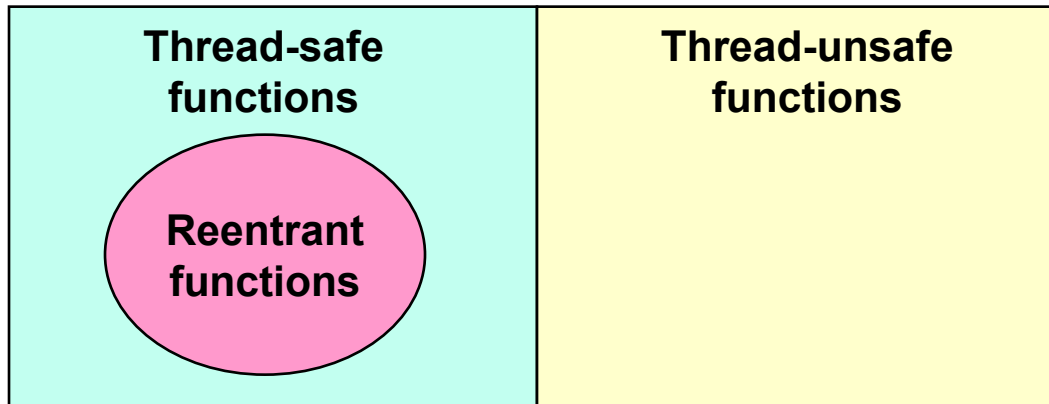
Reentrant functions

- A function is **reentrant** iff it accesses NO shared variables when called from multiple threads
 - Reentrant functions are an important subset of thread-safe functions
 - Require no synchronization operations
 - Otherwise, deadlock may happen (recall signal handler)

Q: Why cannot using printf in a signal handler?

Reentrant functions

All functions



NOTE: The fixes to Class 2 thread-unsafe functions require modifying the function to make it reentrant

Thread local storage: an example

```
thread_local int i=0;
```

```
void f(int newval) {  
    i = newval;  
}
```

```
void g() {  
    std::cout << i;  
}
```

```
void threadfunc(int id) {  
    f(id);  
    ++i;  
    g();  
}
```

```
int main() {  
    i=0;  
    std::thread t1(threadfunc,1);  
    std::thread t2(threadfunc,2);  
    std::thread t3(threadfunc,3);  
  
    t1.join();  
    t2.join();  
    t3.join();  
    std::cout << i << std::endl;  
}
```

Q: What are the possible outputs?

"2340", "3240", "4230", "4320", "2430" or "3420"

TLS: Thread local storage

- Useful for data that are logically global to the thread
 - Good for avoiding passing references to it up and down call stack
 - Lifetime is the lifetime of the thread
- Good fences make good neighbors
 - Not visible to other threads (unless a pointer/reference is given away)

Example-1 of TLS: Random number generator

- If your seed was a local variable within the random function
 - it would be initialized every time you called it, giving you the same number each time
 - If it was a global, threads would interfere with each other's sequences
- Using TLS
 - Each thread gets its own random number sequence, independent of other threads

Example-2: Errno

- How is errno maintained?
 - A global variable
 - Modified by the library after some error happens
- What if another thread has an error at the same time?
 - Must avoid change the global errno
 - Solution: use TLS to save errno

Issues: Races

- Correctness of a program depends on
 - one thread reaching point x in its control before another thread reaches point y

Issues: Races

```
#define N 4
```

```
int main()
```

```
{
```

```
    pthread_t tid[N];
```

```
    int i ;
```

```
    for (i = 0; i < N; i++)
```

```
        pthread_create(&tid[i], NULL, thread, &i);
```

```
    for (i = 0; i < N; i++)
```

```
        pthread_join(tid[i], NULL) ;
```

```
    exit(0) ;
```

```
}
```

```
/*thread routine */
```

```
void *thread(void *vargp)
```

```
{
```

```
    int myid = *((int *)vargp) ;
```

```
    printf("Hello from th. %d\n", myid);
```

```
    return NULL ;
```

```
}
```

Issues: Deadlock

```
int main()
{
    pthread_t tid[2];
    Sem_init(&mutex[0], 0, 1);  /* mutex[0] = 1 */
    Sem_init(&mutex[1], 0, 1);  /* mutex[1] = 1 */
    Pthread_create(&tid[0], NULL, count, (void*)0);
    Pthread_create(&tid[1], NULL, count, (void*)1);
    Pthread_join(tid[0], NULL);
    Pthread_join(tid[1], NULL);
    printf("cnt=%d\n", cnt);
    exit(0);
}
```

Issues: Deadlock

```
void *count(void *vargp)
{
    int i;
    int id = (int) vargp;
    for (i = 0; i < NITERS; i++) {
        P(&mutex[id]); P(&mutex[1-id]);
        cnt++;
        V(&mutex[id]); V(&mutex[1-id]);
    }
    return NULL;
}
```

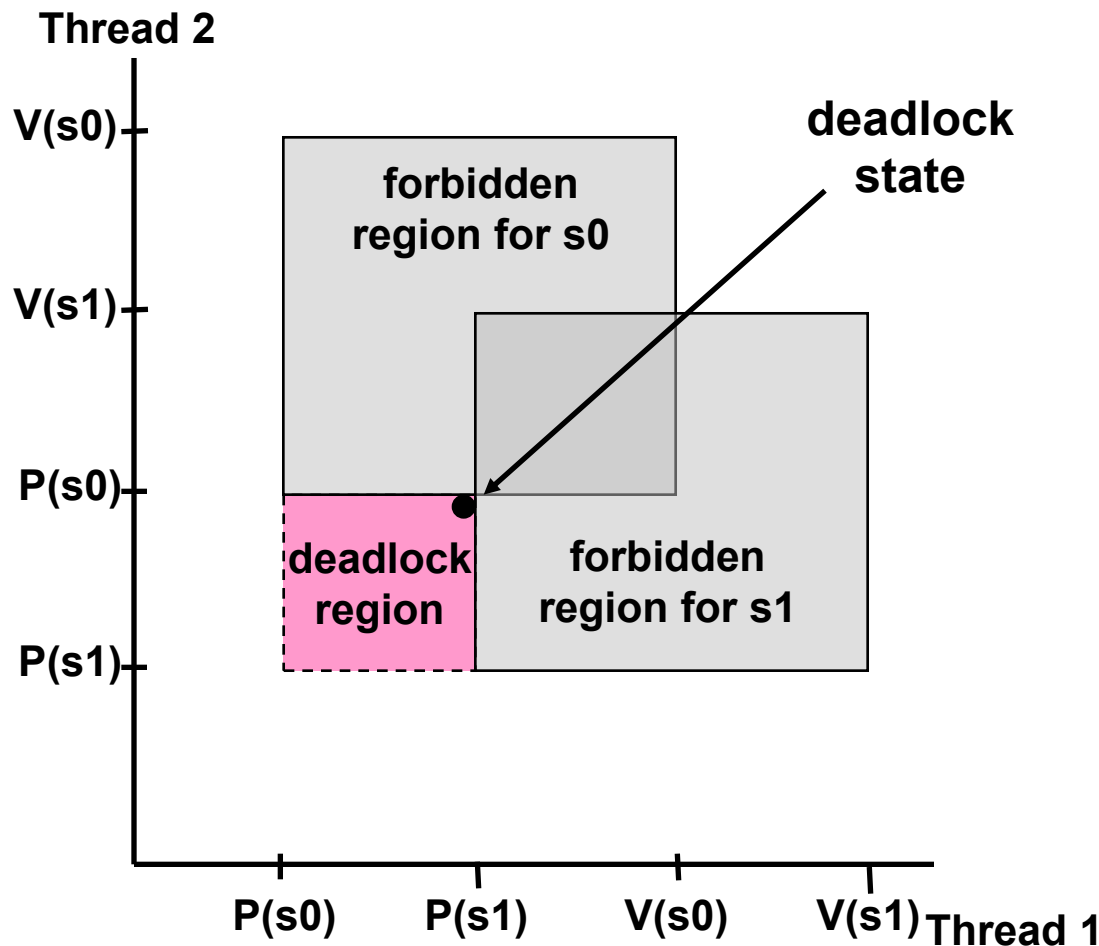
Thread0

Tid[0]:
P(s₀);
P(s₁);
cnt++;
V(s₀);
V(s₁);

Thread1

Tid[1]:
P(s₁);
P(s₀);
cnt++;
V(s₁);
V(s₀);

Issues: Deadlock



Initially, $s=t=1$

Locking introduces the potential for **deadlock**: waiting for a condition that will never be true.

Any trajectory that enters the **deadlock region** will eventually reach the **deadlock state**, waiting for either s or t to become nonzero.

Other trajectories luck out and skirt the deadlock region.

Threads Summary

- Threads provide another mechanism for writing concurrent programs
- Threads are growing in popularity
 - Cheaper than processes
 - Easy to share data between threads
- However, the ease of sharing has a cost:
 - Easy to introduce subtle synchronization errors
 - Tread carefully with threads!